



08 – Roberta Williams

Organización: <https://github.com/UNIZAR-30226-2026-08>

Contacto

845977	Jaime Alconchel Gallego
844699	Mario Casado Fontana
842102	Cristina Embid Martínez
844472	Hugo Naudín López
844459	Miguel Ángel Luquín Guerrero
792310	Julia Quero Pérez
843740	Nicolás Eugenio de Rivas Morillo
844487	Miguel Ángel Royo Miguel

Plan de gestión, análisis, diseño y memoria
del proyecto

MAGNATE

22 de mayo de 2026

Índice

1. Introducción	3
2. Organización del proyecto	4
3. Plan de gestión del proyecto	4
3.1. Inicio del proyecto	4
3.1.1. Mitigación de riesgos	5
3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto	6
4. Despliegue	6
4.1. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento	11
5. Análisis y diseño del sistema	14
5.1. Análisis de requisitos	14
5.2. Diseño del sistema	15
5.2.1. Estrategia de los bots de Magnate	16
5.2.2. Diseño de interfaz de usuario	18
5.2.3. Diseño de elementos del juego	21
5.2.4. Decisiones de diseño	21
6. Memoria del proyecto	23
6.1. Inicio del proyecto	23
6.2. Ejecución y control del proyecto	23
6.3. Cierre del proyecto	23
6.3.1. Dedicación por integrante	24
7. Conclusiones	24
A. Resultados de la revisión intermedia	26
B. Presentación final	28
C. Diagramas	28
C.1. Diagrama de componentes y conectores.	28
C.2. Diagrama de estados de la fase de Subasta de un turno del juego Magnate	29
C.3. Diagramas de módulos	29
D. Análisis de riesgos	31
E. Reglas	33
E.1. Introducción	33
E.2. Objetivo del juego	33
E.3. Equipamiento	33
E.3.1. Los dados: definición y funcionamiento	33
E.3.2. Tablero	34
E.4. Casillas	34
E.4.1. Propiedades	34
E.4.2. Grupos de propiedades	34
E.4.3. Puentes	34

E.4.4. Servidores	34
E.4.5. Casillas fantasía	35
E.4.6. Vas a la cárcel	35
E.4.7. Cárcel	35
E.4.8. Salida	35
E.4.9. Tranvías	35
E.4.10. Párking	35
E.5. Preparación	35
E.6. Turno de juego	36
E.7. Subastas	36
E.8. Alquileres	36
E.9. Construcciones	36
E.10. Intercambios	36
E.11. Bonificaciones finales	37
E.12. Bancarrota	37

1. Introducción

El presente documento recoge el plan de gestión, análisis, diseño y memoria del proyecto Magnate. Magnate es un sistema que consiste en un juego multiplataforma, para escritorio y web, online y multijugador, con un diseño sencillo, estético y lúdico. Magnate es un juego de mesa por turnos basado en el popular juego Monopoly, tomando inspiración adicional de la saga de videojuegos Mario Party. Magnate busca ser un juego dinámico con alto grado de aleatoriedad, pero sin perder la estrategia del Monopoly original, al que añade más reglas que esperan hacer de la experiencia de juego novedosa y emocionante. Estas incluyen un tablero en dos niveles, un nuevo sistema de cartas, más formas de desplazamiento a lo largo del tablero y un nuevo sistema de puntuación.

El sistema permitirá dos modos de juego: partidas públicas y partidas privadas. Las partidas públicas reunirán a 4 usuarios interesados en jugar. Las partidas privadas permitirán el juego de varios jugadores que dispongan del código de la sala y bots, sumando hasta 4 jugadores.

Otros objetivos del sistema son:

- Reanudar partidas entre dispositivos.
- Asociar cuentas de usuario con la información de las partidas.
- Ofrecer una experiencia atractiva y personalizable, incluyendo una tienda de cosméticos.
- Garantizar la seguridad y privacidad de los usuarios.

El sistema será compatible con el navegador web Chromium 144.0.7559.96 y para aplicaciones de escritorio con el sistema operativo Arch Linux 6.18.6. Se desplegará en varios contenedores Docker en un servidor de la empresa 08 – Roberta Williams. Las tecnologías a utilizar abarcan Godot 4.6, React, Django y PostgreSQL.

El desarrollo del sistema se realizará a lo largo de los meses de febrero, marzo, abril y mayo. Los hitos principales son:

- Entrega intermedia: 6 de abril de 2026. Prototipos sin integración.
- Presentación del sistema: 4 de mayo de 2026.
- Muestra del sistema al cliente: 15 de mayo de 2026.
- Entrega final y cierre de proyecto: 27 de mayo de 2026.

El precio del sistema y el proyecto suman 30795,50 €. Por dicho precio se entregará lo siguiente a la finalización del proyecto:

- Todo el código desarrollado.
- Documentación de despliegue y ficheros de configuración y Docker Compose para el despliegue.
- Un AppImage con la versión escritorio, ya compilado.

El cuerpo del documento se estructura como sigue: en la Sección 2 se especifican los integrantes, roles y tareas de la empresa; en la Sección 3 se describe la planificación inicial del proyecto, en cuanto a los recursos necesarios, temporalidad, procedimientos y responsables de las tareas; en la Sección 5 se describe el sistema, enumerando los requisitos, especificando el diseño y tecnologías y justificando las decisiones de diseño tomadas; en la Sección 6 se recoge la memoria del proyecto, y por último en la Sección 7 se recogen las conclusiones y propuestas de mejora del proyecto. Los apéndices incluyen el Apéndice A, que recoge los resultados de la evaluación intermedia; el Apéndice B, con una revisión de la presentación realizada; el Apéndice C, donde se incluyen algunos diagramas; el Apéndice D, con la información complementaria del análisis de riesgos realizado; y el Apéndice E, donde se detallan las reglas del juego Magnate.

2. Organización del proyecto

Los encargados del proyecto son la totalidad de la empresa 08 – Roberta Williams, que conformamos los estudiantes de 5º curso del Programa Conjunto en Matemáticas e Ingeniería Informática de la Universidad de Zaragoza. La creación del equipo de trabajo surge de manera natural tras las experiencias compartidas a lo largo del grado, que refuerzan nuestra confianza y seguridad en nosotros mismos como equipo y como profesionales.

Basándonos en nuestras habilidades y experiencia previa, se ha decidido conjuntamente el siguiente reparto de cargos y tareas principales, recogido en la Tabla 1.

Hugo			IA		
Jaime	backend	despliegue	encargado despliegue	diagramas	coordinador backend
Mario			lógica del juego	diseño de marca	
Miguel Royo	frontend	web	coordinador frontend		diseño interfaz
Cristina			encargada calidad web		
Julia			calidad	documentación	gestión del proyecto
Nicolás		escritorio	encargado escritorio	diagramas	diseño interfaz
Miguel Luquín			encargado calidad escritorio		

Tabla 1: Relación de cargos y tareas principales en la empresa 08 – Roberta Williams.

De los roles recogidos en la Tabla 1, queremos destacar las siguientes responsabilidades:

- Directora de proyecto: Julia Quero Pérez.
- Coordinador de frontend: Miguel Ángel Royo Miguel.
- Coordinador de backend: Jaime Alconchel Gallego.
- Coordinadora de calidad: Cristina Embid Martínez.
- Coordinador de despliegue: Jaime Alconchel Gallego.

3. Plan de gestión del proyecto

En esta sección se describe cómo se llevarán a cabo los distintos procesos o tareas a realizar a lo largo del tiempo, su temporalidad, los recursos a emplear y quiénes son sus responsables.

3.1. Inicio del proyecto

Se ha priorizado el uso de herramientas abiertas o de software libre para la realización del proyecto, por lo que los recursos materiales y software se reducen a los ya accesibles a los miembros del equipo. Estos incluyen los ordenadores portátiles de cada miembro del equipo y el servidor que posee Mario.

La formación inicial necesaria para los miembros del equipo, por tecnología utilizada, se detalla a continuación:

- Godot 4.6 (implementación del sistema para aplicación de escritorio): Miguel, Miguel y Nicolás buscarán por su cuenta tutoriales de la aplicación.
- React (implementación del sistema para navegador web): Miguel Royo buscará información al respecto, y Cristina le ayudará a buscar tutoriales y proyectos de la librería Phaser para el juego. Julia se encargará de incluir el sonido.
- Django (backend): Los miembros del equipo ya tenían experiencia con la tecnología.
- PostgreSQL (gestor de BDD): Los miembros el equipo ya tenían experiencia con la tecnología.

3.1.1. Mitigación de riesgos

Se han detectado los siguientes riesgos del proyecto:

- Ri1 Hay integrantes del equipo que tienen muchos proyectos (y/o asignaturas a la vez).
- Ri2 La falta de experiencia trabajando juntos puede afectar al éxito del proyecto.
- Ri3 La toma de decisiones se puede bloquear por no estar dispuestos los miembros del equipo a asumir las responsabilidades.
- Ri4 Nunca se ha trabajado en un grupo tan grande.
- Ri5 El desconocimiento del cliente.
- Ri6 El proyecto cuenta con unas funcionalidades que no están muy claramente definidas.
- Ri7 Nunca se ha construido un sistema tan complejo como el propuesto.
- Ri8 Se ha elegido una tecnología que se conoce poco o nada.
- Ri9 Se ha trabajado, o no, con el entorno de despliegue elegido.
- Ri10 La fecha de entrega está muy lejos en el tiempo.
- Ri11 El proyecto hay que desarrollarlo en dos fases.

Para ellos se proponen las siguientes estrategias de mitigación:

- Ri1 Promover la organización individual, evitar el solapamiento en la definición de las tareas y fijar fechas internas para todas las pequeñas tareas. Responsables: Julia, Cristina.
- Ri2 Favorecer la comunicación inmediata en los grupos de WhatsApp y Discord. Responsable: Julia.
- Ri3 Repartir las tareas conforme surgen. Responsables: Julia, Jaime y Miguel Royo.
- Ri4 Fomentar el uso de herramientas y metodologías para la comunicación fluida (WhatsApp, Discord), realizar un reparto de trabajo a nivel de los grupos de trabajo y posteriormente ponerlo en común y conjunto en las sesiones de prácticas con el equipo al completo. Responsables: Julia, Jaime y Miguel Royo.
- Ri5 Aprovechar las sesiones de prácticas para consultar el estado del proyecto y solicitar retroalimentación. Responsable: Julia.
- Ri6 Establecer requisitos claros. Responsable: Julia.

- Ri7 Dividir el proyecto en tareas pequeñas y autocontenidas por equipos y establecer fechas para juntar los resultados. Responsable: Julia.
- Ri8 Fomentar la consulta y realización de tutoriales al comienzo del proyecto, previo al desarrollo. Responsable: Miguel Luquín.
- Ri9 Revisar proyectos ya realizados con esa misma tecnología y hacer alguna prueba menor antes de configurar el sistema completo. Responsable: Jaime.
- Ri10 Poner fechas internas con un margen suficiente para entregar a tiempo el proyecto en las fechas de entrega con el cliente. Responsable: Julia.
- Ri11 Establecer fechas intermedias (hitos en Github issues). Responsables: Cristina, Nicolás, Mario.

3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto

Las **herramientas** a utilizar en el proyecto se han elegido por ser aquellas que aportan más profesionalidad y por contar cada miembro con experiencia en su uso.

El código del proyecto se ha desarrollado en el entorno de desarrollo (VisualStudio Code o vim) de la preferencia de cada integrante, y se ha integrado en la plataforma GitHub, utilizando la herramienta git de control de versiones, así como la documentación escrita con LaTeX.

Se ha utilizado la plataforma de gestión de archivos Google Drive y sus filiales Google Sheets y Google Docs para el seguimiento del desempeño, la realización de presupuestos o análisis y compartir de forma preliminar cierta información bibliográfica o de gestión.

Para la creación de diagramas se ha seguido la notación UML, y se ha utilizado PlantUML. Para el diagrama de Gantt, por otro lado, se ha utilizado una hoja de cálculo de Google Sheets.

Para el diseño de la GUI, en primer lugar se concretó una guía de estilo general de los elementos de la interfaz, y posteriormente se realizan a papel los bocetos de cada pantalla, que tienen su primer prototipo directamente en la tecnología escritorio o web utilizada en el proyecto. Los diseños se hacen siguiendo el sentido estético del equipo de frontend, más desarrollados por algunos de sus miembros que otros, que se centran en las cuestiones de usabilidad. Como referencia para la interfaz se siguen Las 8 reglas de oro de Schneiderman, y se intentan minimizar la interacción del usuario sin comprometer su comprensión. Para la guía de estilo se eligieron un número reducido de colores principales y secundarios y se consensuaron los formatos de los elementos básicos (botones, fondos...) para la consistencia.

Por último, las herramientas de despliegue engloban, a nivel software, contenedores Docker que desplieguen un sistema operativo Alpine Linux; y a nivel hardware, para el despliegue de los contenedores, una máquina propiedad de Mario Casado con las siguientes especificaciones: UGREEN DXP2800 con procesador Intel n100, 8TB de HDD, 1TB de SSD m2 y 16GB de RAM DDR5. Como todo el despliegue se realizará en la misma máquina, los puertos y rutas necesarios para el despliegue aparecerán como variables de entorno en los ficheros yaml, y no será necesario especificar nada más.

4. Despliegue

Se ha optado por desarrollar un modelo de despliegue agnóstico, adaptable y escalable mediante contenedores Docker sincronizados por el software docker-compose. Cada uno de los servicios necesarios para realizar el despliegue se configura como un contenedor en la red interna virtual creada por esta herramienta. Para sincronizar el contenido de los tres repositorios de los que se servirá el entorno desplegado, se ha utilizado git y diseñado un repositorio específico de

despliegue, aprovechando su soporte para submódulos para mantener actualizados los distintos repositorios del proyecto. Así, el repositorio creado presenta la siguiente estructura:

Fichero / Directorio	Descripción
<code>docker-compose.yml</code>	Fichero que describe la construcción y configuración de todos los contenedores.
<code>Dockerfile.backend</code>	Dockerfile para empaquetar y crear el contenedor de backend.
<code>Dockerfile.frontend-web</code>	Dockerfile para compilar y configurar el servidor de frontend web.
<code>nginx.conf</code>	Fichero de configuración del servidor web.
<code>build-desktop.sh</code>	Script que compila los binarios del cliente de escritorio en un contenedor aislado de docker y las deposita en <code>desktop-releases/</code> .
<code>generate-certs.sh</code>	Script que genera certificados autofirmados para el servidor web.
<code>backend/</code>	Repositorio fuente de backend (submódulo).
<code>frontend-web/</code>	Repositorio fuente de frontend web (submódulo).
<code>frontend-desktop/</code>	Repositorio fuente de frontend escritorio (submódulo).
<code>.env</code>	Archivo local que contiene las variables de configuración específicas para el despliegue.

Figura 1: Estructura del repositorio de despliegue

Debido a la naturaleza particular de la compilación de godot, se ha escogido separar la creación de los binarios del cliente del juego del sistema `docker-compose`, por lo que ahora es un script específico el que se encarga de generar estos binarios y depositarlos en una carpeta de lanzamientos que luego se expondrá públicamente a través del servidor web. En caso de no realizarse la compilación, el sistema seguiría funcionando, pero los enlaces de descarga estarían vacíos.

Quizás la ventaja más interesante del despliegue desarrollado es que aísla todas las variables de configuración específicas del sistema en el que se va a ejecutar el software (en nuestro caso, un servidor doméstico) de la configuración global, por lo que el cliente únicamente tendría que generar un archivo entorno propio para adaptar el sistema a sus servidores. Por tanto, puede concluirse que el sistema de despliegue desarrollado está preparado para cualquier sistema con soporte para docker (en la práctica, todos los sistemas utilizados en la industria) e incluso se extendería fácilmente a entornos distribuidos, ya que la traducción de `docker-compose` a kubernetes es muy común y relativamente sencilla de realizar gracias a herramientas automáticas como Kompose.

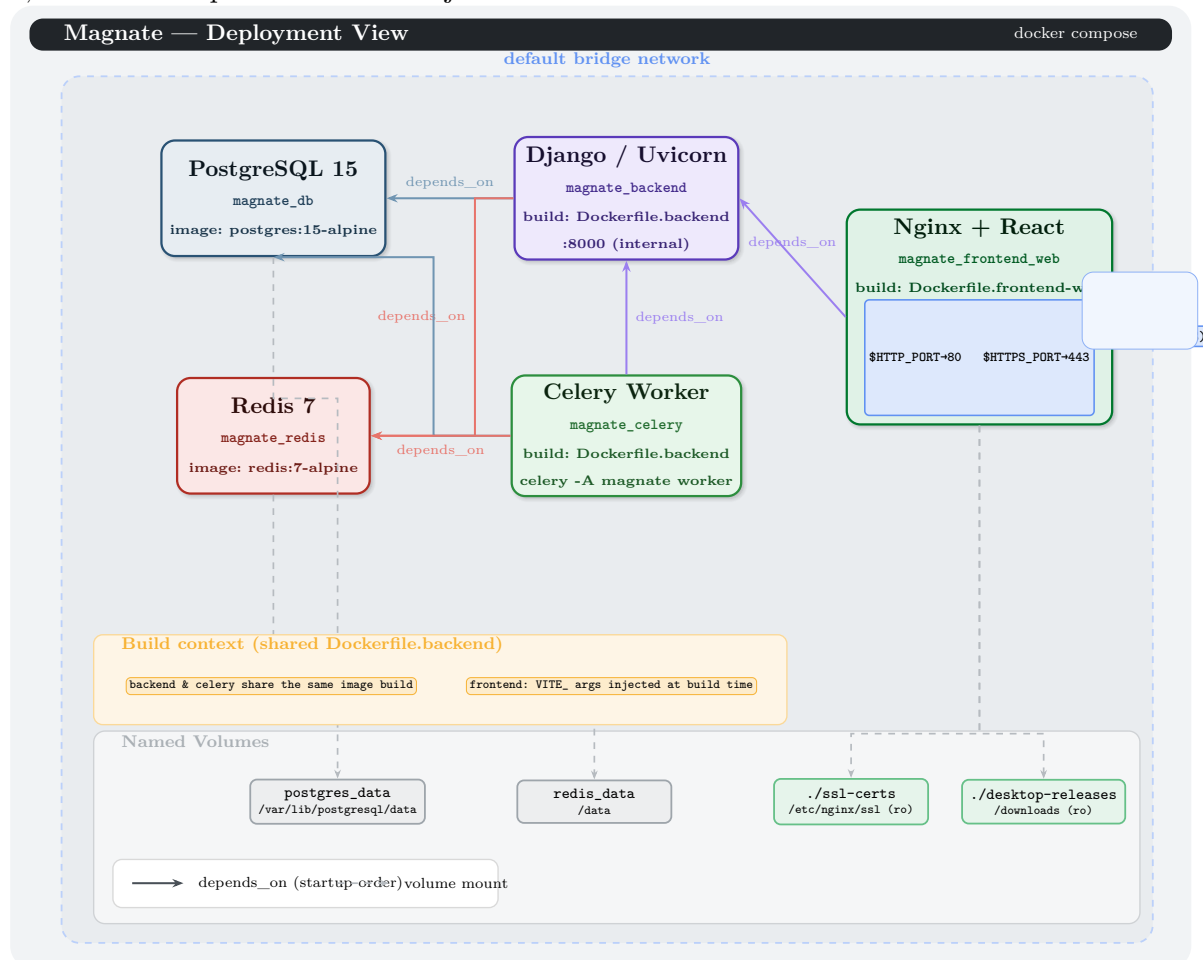
Los sistemas escogidos para los contenedores son los siguientes:

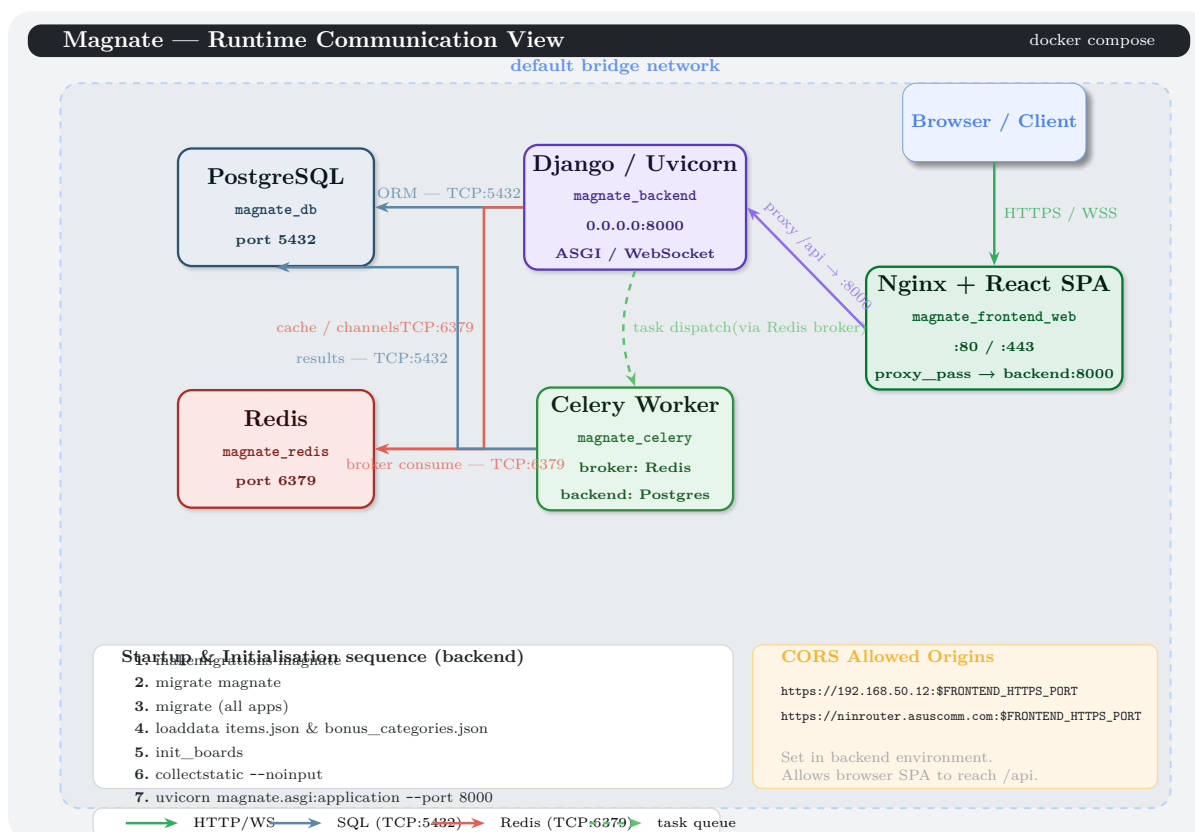
- Para la base de datos, el contenedor específico de PostgreSQL basado en Alpine Linux, por su robustez, uso en el sector y facilidad de configuración (basta con especificar 3 variables de entorno para lograr un despliegue completo).
- Para el servidor redis, se usa directamente el contenedor específico para este software basado en Alpine Linux.
- Para los contenedores de celery y django, utilizamos la imagen oficial de Python.
- Para la compilación del código de frontend web, se usa el contenedor oficial de NPM con base Alpine.
- Para el servidor web, se usa la imagen oficial de NGINX basada en Alpine Linux.

- Para la compilación de los clientes de escritorio, se usa un contenedor no oficial de Godot (pero verificado y actualizado) basado en Debian.

En general, se observa que casi todos los contenedores están basados en Alpine Linux. Esto no es casualidad, ya que esta es una distribución extremadamente popular en contenedores Docker debido a su sencillez, reducido tamaño, buen rendimiento y seguridad.

Finalmente, incluimos un diagrama de despliegue con los distintos contenedores que se utilizan, tanto en compilación como en ejecución:





Los **responsables** de supervisar la corrección de las configuraciones, construcción del software y calidad del producto son para el código del backend Jaime (el coordinador del backend) y para el código del frontend Miguel (el coordinador del frontend).

La convención de nombrar ficheros y documentos y **estándares de código** se sigue según los estándares de los lenguajes empleados, y según las especificaciones de las herramientas de generación automática de documentación empleadas. A nivel general y común a todos ellos, puede especificarse que el código está escrito en inglés (nombres de variables y comentarios), utilizando el nombrado con camelCase para las clases y el resto de variables con snake_case. Para la generación de la documentación se contemplaron distintas herramientas para cada módulo (backend, web, escritorio). Para los módulos del backend: el código interno y los websockets utilizan mkdocs con docstrings, y el código de API Rest utiliza swagger. Para los módulos del cliente web se utiliza TypeDoc. Para los módulos del cliente escritorio se utiliza Godot (ver [4]).

Para el desarrollo de la aplicación para entornos de escritorio se seguirán las buenas prácticas explicadas en [2]. Entre estas, se destacan algunos detalles específicos a las herramientas:

- Signal up call down: Aplicar el patrón de diseño Publicación-Subscripción mediante señales de Godot, manteniendo la jerarquía de nodos.
- Estilo: El lenguaje de programación de la herramienta, GDScript, es muy similar a Python, se recomienda el uso del estándar PEP8 (ver [5]).
- Evitar clases pesadas: Se heredarà de clases más ligeras que la clase Node, como RefCounted.

El código se ha organizado en 4 **repositorios** de GitHub: uno para el backend, otro para el frontend del cliente web, otro para el frontend del cliente escritorio y otro para la gestión y documentación. Todos los miembros del equipo tienen acceso a todos los repositorios, y respetan el workflow consensuado en el equipo. Las issues se crean tras una reunión entre los miembros

del equipo correspondiente en la que se identifican, definen y reparten las tareas a realizar. Se designa un encargado que cree en Github las issues correspondientes. Para realizar un commit en el repositorio compartido, se ha de obtener el permiso de los demás usuarios habituales del repositorio (los miembros del equipo correspondiente) para minimizar las posibles incongruencias (aunque ya se definen las tareas teniendo esto en cuenta), y no se han de revisar los commits ya que se confía en la capacidad y buen criterio de los miembros del equipo.

Ante la detección de **incidencias**, el que la haya detectado tiene la responsabilidad de encontrar a la persona que haya generado la incidencia, que llamaremos a partir de ahora el incidente. Será responsabilidad del incidente decidir el plan de actuación para solventar la incidencia, en tanto a si puede ser el único responsable y si es más prioritario o no que el trabajo actual que haya que realizar.

Las actividades de **control de calidad** serán variadas. Se realizarán pruebas por separado de frontend y backend y posteriormente se realizarán pruebas de la conjunción de ambos. Para las pruebas del backend se cuenta en primera instancia con tests para comprobar el correcto funcionamiento de las funciones. Para probar el juego completo, se propone:

1. Probar todos los arcos del grafo de juego forzando los escenarios correspondientes y eliminando el azar.
2. Simular partidas con dos agentes jugando entre ellos.
3. Prueba de los serializadores.

Para las pruebas del frontend,

1. En primera instancia, se comprobará manualmente la correcta inclusión del código en el sistema existente.
2. Se documentará mínimamente las entradas y salidas esperadas para la interacción con el juego (tirada de dados, compra de propiedad, pago de alquiler...) y se comprobará sistemáticamente la corrección del cliente correspondiente partiendo de un ejemplo de partida.

Para las pruebas de integración, además de probar lo anteriormente especificado, tanto por separado como con pruebas conjuntas, se realizarán las siguientes pruebas:

1. Prueba de websockets.
2. Pruebas con cliente manual de websockets para simular la interacción del frontend.

Las pruebas de despliegue consistirán en realizar en el sistema en el que se realizará finalmente el despliegue todas las pruebas mencionadas anteriormente. Para el código a alto nivel, se contrastará con la persona que realice los diagramas y los autores del código correspondiente que el comportamiento del sistema coincide con lo expresado en el diagrama. Tanto la realización de las pruebas como el despliegue del sistema se realizará durante días designados para la integración entre backend y frontend, como expresado en el diagrama de Gantt de la Figura 2. Se especificarán las fechas concretas para realizar las actividades al comienzo de la semana correspondiente a estas actividades. Por tanto, descartamos tanto metodologías de integración como de despliegue continuos.

Se entregará al cliente lo siguiente a la finalización del proyecto:

- Todo el código desarrollado.
- Documentación de despliegue y ficheros de configuración y Docker Compose para el despliegue.
- Un AppImagen con la versión escritorio, ya compilado.

4.1. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento

La responsable del calendario del proyecto, coordinación, y comunicaciones es Julia, la directora del proyecto, apoyada en los coordinadores backend y frontend para una mejor descripción y ordenación de las tareas, la división del trabajo y la monitorización y seguimiento de las tareas.

Para la **monitorización y medición del progreso**, cada lunes y viernes se revisarán las tareas realizadas por los miembros del equipo correspondiente. Se mira la consecución de la tarea, y de haber quedado mucho por realizar, dado que confiamos en el buen trabajo de los miembros del equipo, concluiremos que el retraso se ha debido a un reparto no equitativo y se volverá a repartir la tarea faltante entre los miembros del equipo, para realizarse en esa semana, antes que las tareas de dicha semana.

Se ha intentado realizar un plan lo suficientemente holgado como para que estas desviaciones, de no prolongarse durante más de una semana, no supongan un problema. De darse continuamente este retraso, a una semana vista de la entrega se replantearán las tareas a realizar, minimizando estas.

Sobre el **reparto y definición de tareas**, primero se ha realizado una planificación global por requisitos como expresado en el diagrama de Gantt de la Figura 2. Para ello, se ha seguido un planteamiento incremental, encargándonos primeros de implementación, después de integración y por último de despliegue, realizando actividades de calidad tras cada una de estas fases. A más bajo nivel, las tareas se concretarán y realizarán después de la monitorización del progreso, listando el desglose de tareas necesarias para conseguir los objetivos semanales correspondientes al diagrama de Gantt de la Figura 2. Los responsables de las tareas son como lo descrito en la Tabla 1. Del diagrama de Gantt podemos destacar los siguientes hitos internos:

- Entrega intermedia: 6 de abril de 2026. Para esta no se planea haber conseguido completar ningún requisito, pero sí tener listo a falta de integración y despliegue la implementación correspondiente a los requisitos 1, 2, 3, 4, 6 y 9.
- Comienzo de la integración: 3 de abril de 2026.
- Comienzo del despliegue: 15 de abril de 2026.
- Presentación del sistema: 5 de mayo de 2026.
- Muestra del sistema al cliente: 15 de mayo de 2026.
- Entrega final: 27 de mayo de 2026.

Una vez se hayan desglosado las tareas y concordado cómo se van a realizar (todo aquello que suponga cohesión, como interfaces consistentes o comunicación entre elementos), se repartirán entre los miembros del equipo (backend, web y escritorio), primando la equidad, los conocimientos y habilidades de los miembros, y sus preferencias. Cada uno informará de sus preferencias y al llegar al consenso entre todos se fijarán las tareas con su responsable como Github issues.

Para la **comunicación interna**, se han creado grupos de WhatsApp para la empresa al completo, backend, frontend, frontend web y frontend escritorio, así como un canal de Discord análogo para comunicaciones y cuestiones de rápida resolución o inmediatas. Se realizarán reuniones de seguimiento entre todos los equipos durante las sesiones de prácticas, así como las reuniones dentro del equipo cuando los miembros lo estimen oportuno. Las reuniones de las sesiones de prácticas servirán para actualizar los planes en caso de desviarse del inicial. Las decisiones tomadas acabarán reflejadas en Github issues o en la documentación (o en la carpeta compartida de Drive a modo de borrador, de ser algo extenso, de forma provisional hasta aparecer en la documentación), por lo que no se utilizan documentos adicionales con actas.

Ante diferencias en la **gestión del equipo humano**, se plantea la inclusión de mediadores imparciales o no involucrados en la disputa inicial para intentar consensuar a las partes. En caso

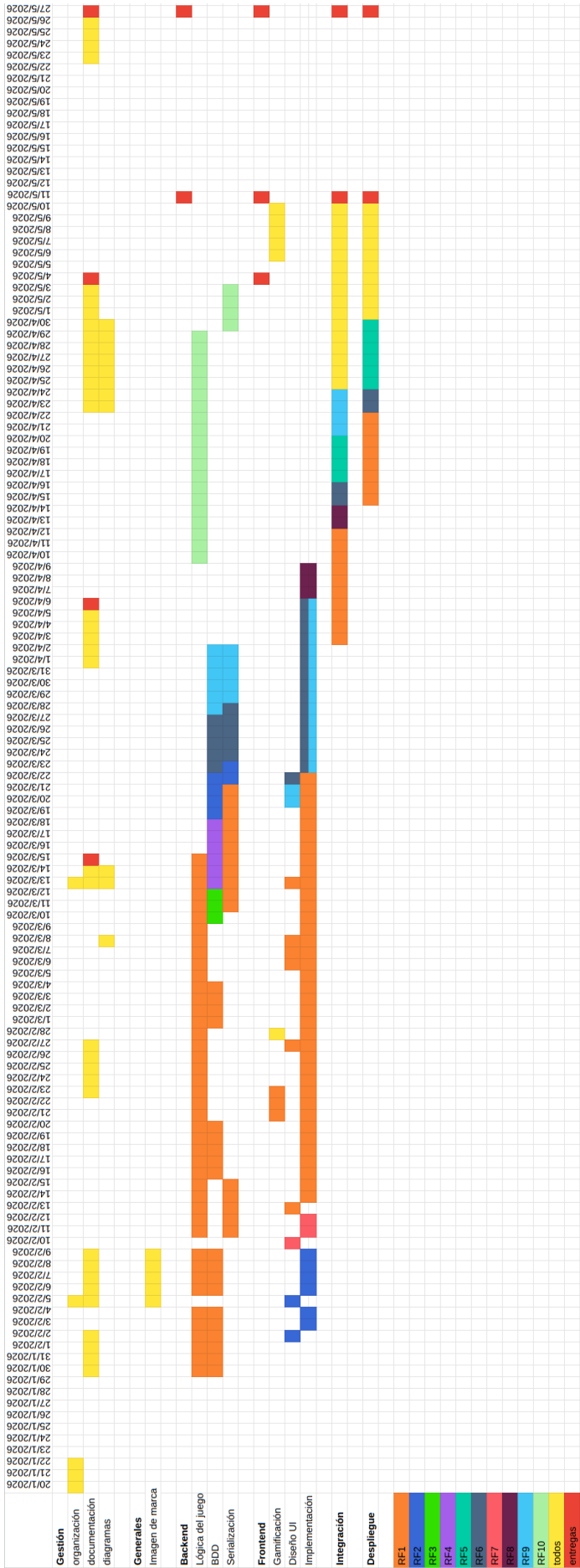


Figura 2: Diagrama de Gantt con la planificación del proyecto.

de no resolverse el conflicto, se pospondrá la discusión a cuando se haya enfriado, no siendo más de 2 semanas.

5. Análisis y diseño del sistema

En esta sección se detallan los requisitos y diseño del sistema.

5.1. Análisis de requisitos

En la Tabla 2 se recoge la relación de requisitos funcionales del sistema, y en la Tabla 3 se recogen los requisitos no funcionales del sistema.

ID	Descripción	Prioridad
RF-1.1	El sistema permitirá jugar a Magnate, siguiendo las reglas descritas en el Apéndice E.	M
RF-1.2	El sistema mostrará los resultados de una partida una vez finalizada, incluyendo el capital final de los jugadores y los beneficiarios de los bonus finales aplicados.	M
RF-2.1	El sistema debe permitir a los usuarios registrarse con un nombre de usuario único y una contraseña.	M
RF-2.2	El sistema exige para el registro de una cuenta de usuario nueva que se confirme la contraseña.	M
RF-2.3	El sistema debe permitir a los usuarios iniciar sesión introduciendo su nombre de usuario y su contraseña.	M
RF-3	El sistema guardará la información de las partidas en curso necesaria para reanudarlas.	M
RF-4.1	El sistema almacenará el historial de partidas de los usuarios.	M
RF-4.2	El sistema permitirá a los usuarios consultar su historial de partidas completas.	M
RF-4.3	El historial de partidas recogerá para cada partida la posición en la que quedó el usuario tras jugarla y las monedas que ganó.	M
RF-5	Las partidas se pueden pausar y reanudar en otra plataforma. Si se está jugando en una plataforma y el usuario se conecta en otra, aun sin haber pausado la partida en la primera plataforma, la partida pasará a jugarse únicamente en la segunda plataforma empleada.	M
RF-6.1	El sistema permitirá a los usuarios que hayan iniciado sesión observar su propia información ligada a un usuario.	M
RF-6.2	La información ligada a un usuario incluye: nombre de usuario, skin de ficha predeterminada para jugar y estadísticas de sus partidas.	M
RF-6.3	Las estadísticas de las partidas ligadas a un usuario incluyen: n ^o partidas jugadas, n ^o partidas vencidas, monedas para la tienda.	M
RF-6.4	El sistema permitirá al usuario elegir su skin de ficha predeterminada entre las que posea.	M
RF-7.1	El sistema permite jugar partidas en dos modalidades: partida pública y partida privada.	S
RF-7.2	Las partidas privadas permitirán que jueguen los poseedores de un código de la sala privada y/o con bots, sumando hasta 4 jugadores.	S
RF-7.3	Las partidas públicas realizan un emparejamiento entre los usuarios por orden de llegada, agrupando 4 jugadores siempre que haya suficientes en línea.	S
RF-7.4	La moneda del juego se recibe tras jugar cada partida como el capital remanente al terminar la partida y aplicar los bonus.	S
RF-8.1	El sistema permitirá a los usuarios intercambiar mensajes en lenguaje natural durante el transcurso de una partida.	C

RF-8.2	El sistema permitirá a los usuarios incluir emoticonos en los mensajes del chat durante el transcurso de una partida.	C
RF-9.1	Se pueden adquirir distintos skins para las fichas del juego en la tienda.	S
RF-9.2	El sistema permitirá a los usuarios adquirir distintos skins para las fichas de las partidas de Magnate en la tienda, intercambiando la moneda del juego, que se obtiene jugando partidas. Habrá al menos 6 fichas distintas, incluyendo una que se otorga por defecto a los jugadores.	S
RF-9.3	Se pueden adquirir distintos emoticonos para emplear en el chat del juego en la tienda.	C
RF-9.4	El sistema permitirá a los usuarios adquirir distintos emoticonos para el chat de las partidas de Magnate en la tienda, intercambiando la moneda del juego, que se obtiene jugando partidas. Habrá al menos 5 emoticonos distintos. Por defecto los jugadores no tendrán ninguno.	C
RF-10	Se diseñará una IA o bots contra la que jugar en las partidas privadas. Este bot tendrá especificadas las reglas del juego, y tomará decisiones aplicando la estrategia establecida en la Sección 5.2.1.	C

Tabla 2: Requisitos funcionales del sistema.

RNF-1	El cliente de escritorio debe funcionar en un computador con sistema operativo Arch Linux 6.18.6.
RNF-2	El cliente web debe funcionar en un navegador con versión Chromium 144.0.7559.96, en una resolución de pantalla de 1920x1080px.
RNF-3	El sistema es seguro a usos malintencionados de las plataformas.

Tabla 3: Requisitos no funcionales del sistema.

5.2. Diseño del sistema

El sistema ofrece un cliente web y un cliente escritorio, para los que utiliza una arquitectura en 4 y 3 niveles, como puede verse en el diagrama simplificado de la Figura 3.

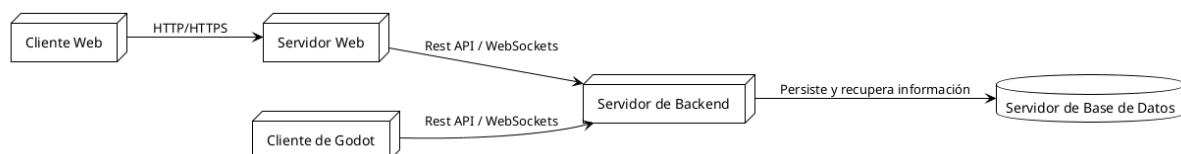


Figura 3: Diagrama de la arquitectura a alto nivel.

El sistema cuenta con un servidor de bases de datos PostgreSQL, un servidor con el backend para la lógica del juego y demás peticiones, y para el caso del cliente escritorio, basta con un único cliente, mientras que para la web necesita un servidor web y sobre este el cliente web con nuestra aplicación. El diagrama de componentes y conectores de la Figura 12 (ver Apéndice C.1) nos concreta el diagrama de la Figura 3 y las comunicaciones entre los elementos desplegados. La comunicación se realiza a través de web sockets para el juego y de API Rest para el resto de la aplicación.

Además aportamos, para explicar la lógica y secuenciación del juego, el diagrama de estados de la Figura 4, que representa un turno de juego y sus fases. El turno comienza con la fase de tirar los dados y después de resolverse lo ocurrido en la casilla a la que se cayó se pasa a la fase de negocios, en la que pueden realizarse opcionalmente y en cualquier orden las transacciones del

juego (rendirse, intercambio, y administrar propiedades: construcción, derruir casas e hipotecar), y por último hay una fase de liquidación análoga a la anterior para que el jugador evite quedarse en números rojos. Los diagramas de secuencia de estas dos últimas fases se encuentran en la Figura 5. Estos muestran la transacción más general (el intercambio), en la que proponente y receptor son distintos.

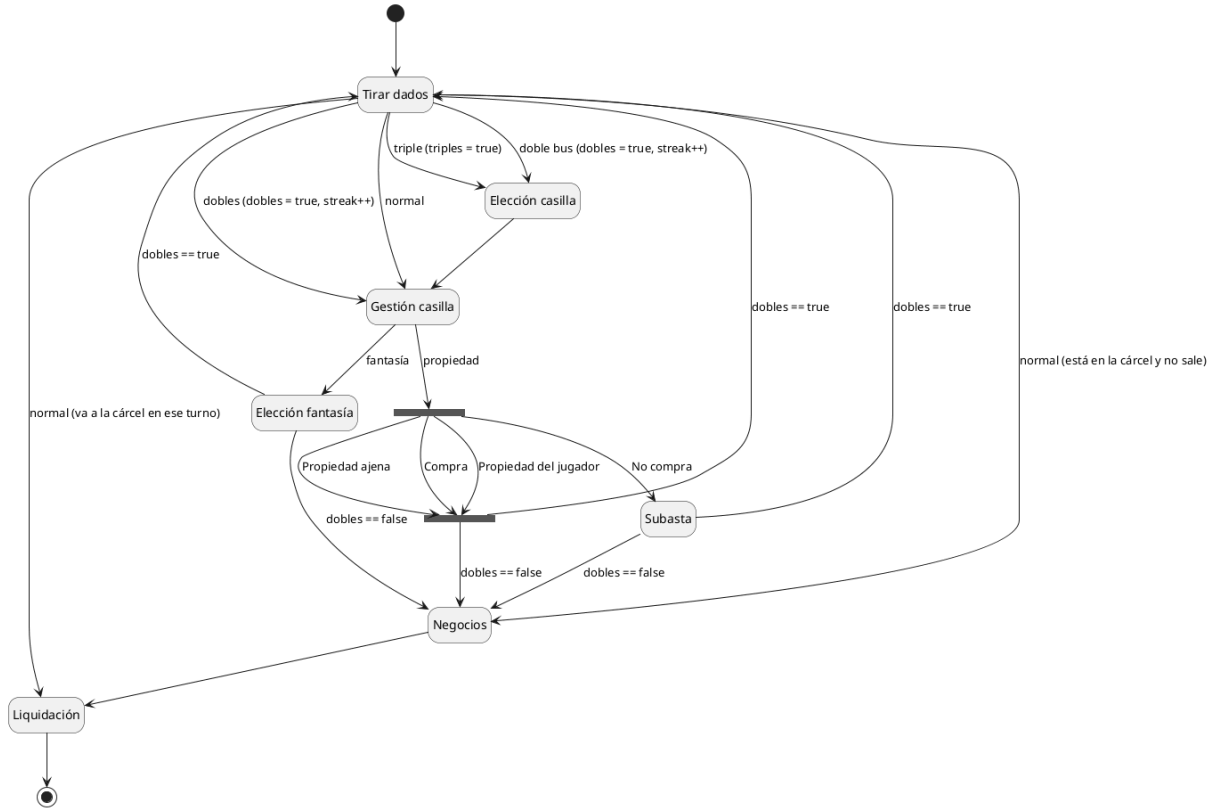


Figura 4: Diagrama de estados de un turno de juego con sus fases.

Otro aspecto del turno por describir es la subasta, en la que se esperará a que todos los jugadores seleccionen el valor por el que subastar (o no participar) para continuar. El diagrama de estados es muy sencillo, se recoge en la Figura 13 (ver Apéndice C.2).

El proyecto está organizado en 3 grandes repositorios de código: el backend, el cliente escritorio y el cliente web. La estructura de estos repositorios, en cuanto a diagramas y vistas de módulos, siguen la estructura habitual de proyectos con estas tecnologías, y pueden consultarse en los diagramas de las Figuras 14, 15 y 16, respectivamente (ver Apéndice C.3).

El despliegue del sistema se realizará en una misma máquina, utilizando varios contenedores, con un sistema en dos capas, sin perjuicio a que se pudiera realizar en varias máquinas distintas cada contenedor, en cuatro capas, según lo especificado en los diagramas de despliegue de la Figura 6.

5.2.1. Estrategia de los bots de Magnate

Los bots que jueguen a Magnate seguirán una estrategia basada en heurísticas. Para ello, tomarán el máximo de las opciones en cada escenario (numerados con números positivos) según los valores esperados que calculen según lo descrito en el documento [1].

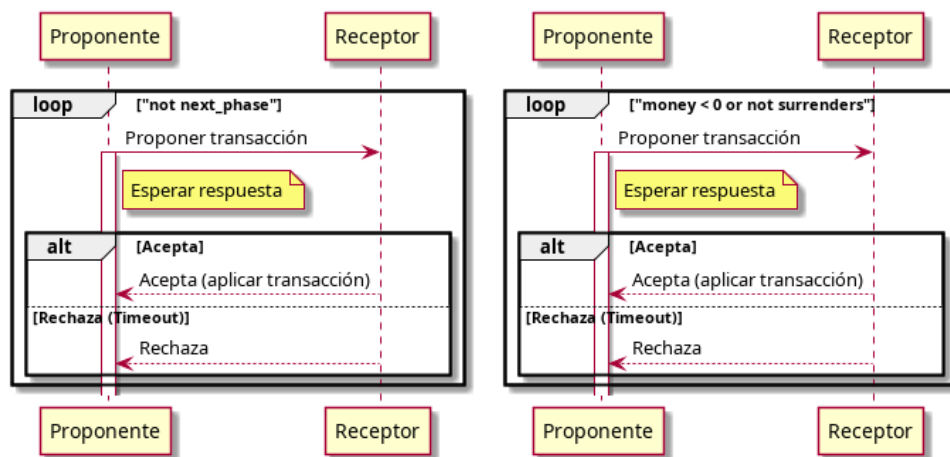


Figura 5: Diagramas de secuencia de las fases de negocios (izquierda) y liquidación (derecha).

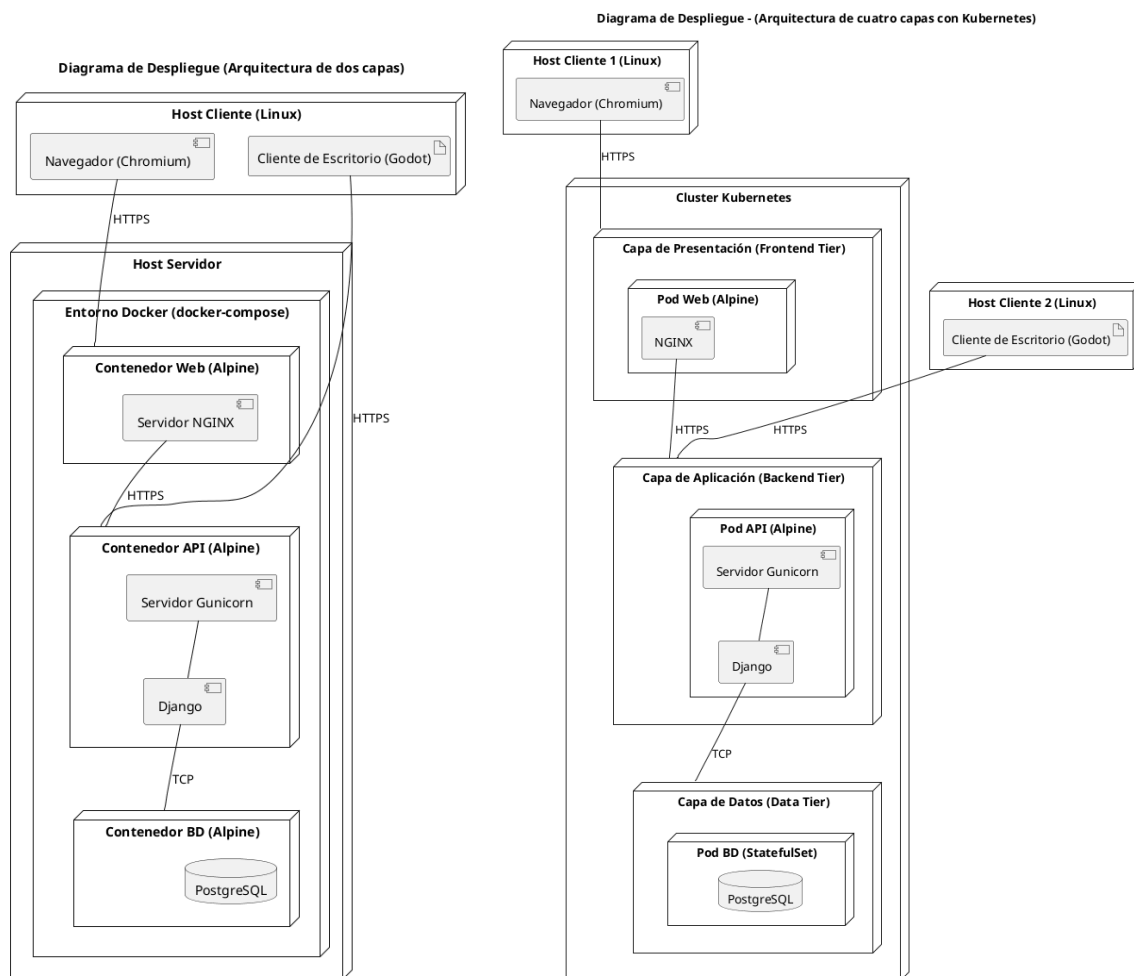


Figura 6: Diagramas de despliegue. A la izquierda, diagrama en 2 capas, como se realizará el despliegue, y a la derecha, en 4 capas, señalando las posibilidades de despliegue.

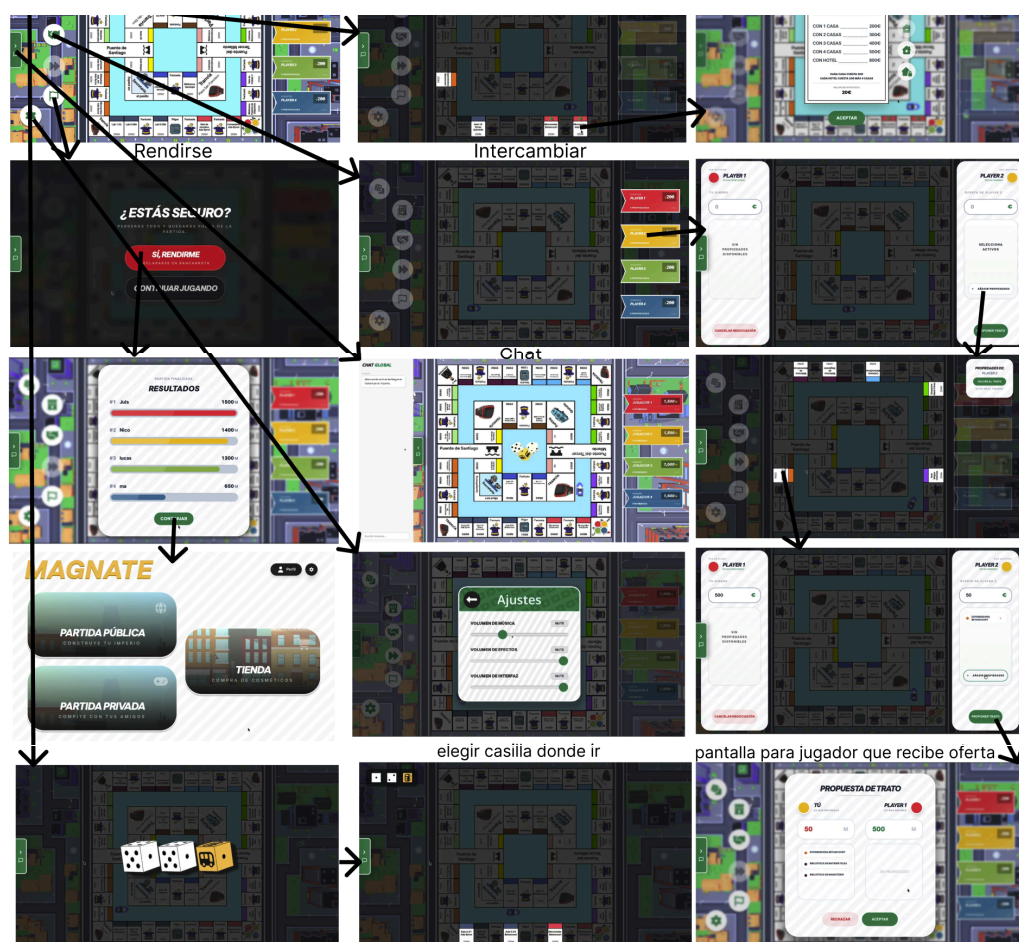


Figura 8: Mapa de navegación (juego, acciones y negocios).

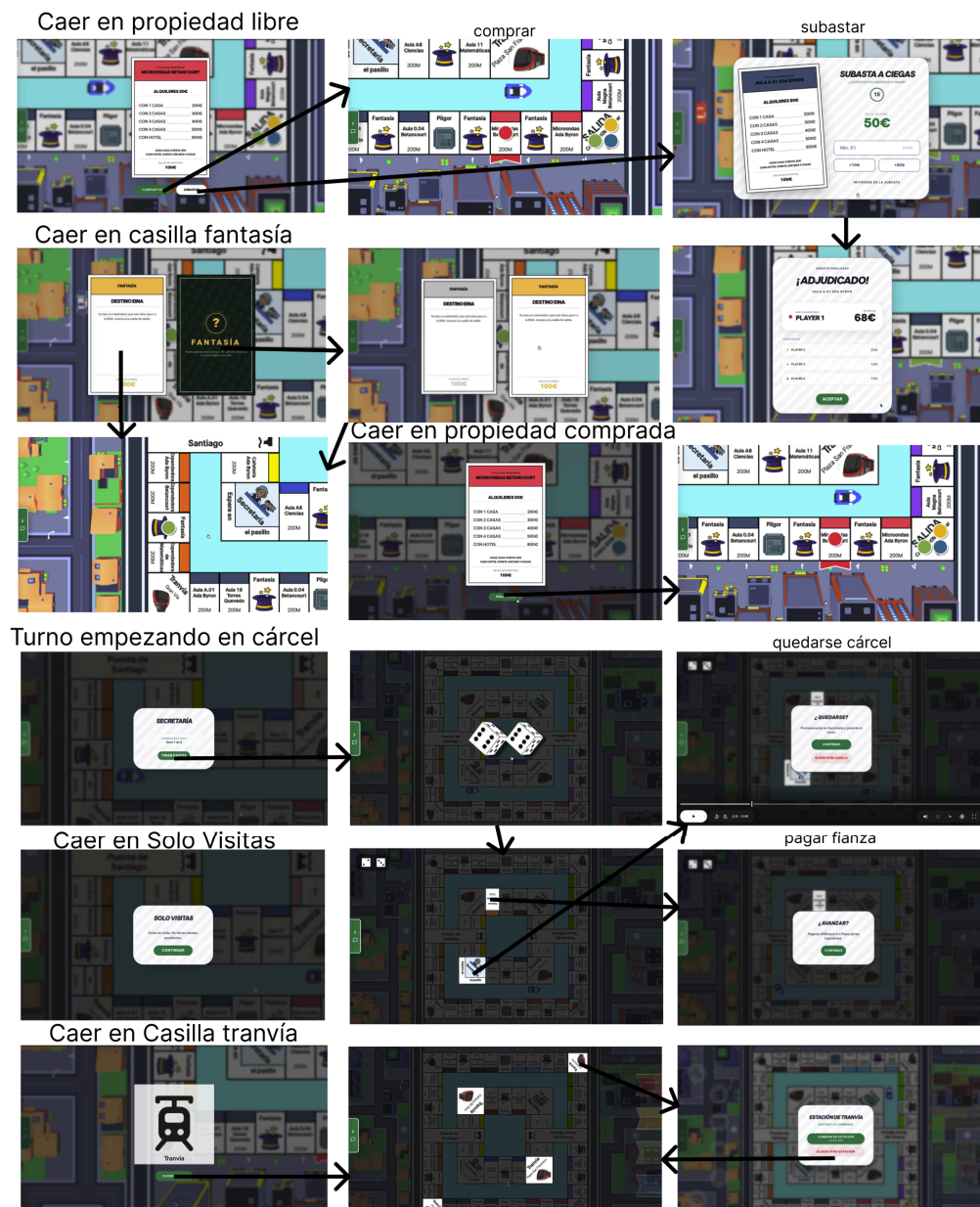


Figura 9: Mapa de navegación (juego, caer en casillas y flujos).

5.2.3. Diseño de elementos del juego

En esta sección concretamos el diseño de algunos skins o emoticonos de la tienda como ejemplo.

Los emoticonos a usar en el chat pueden encontrarse en [6], y los skins de las fichas de las partidas serán diseños en 3D que tomen el color que tenga el jugador en la partida. Puede verse algún diseño de skin de ficha en la Figura 10.

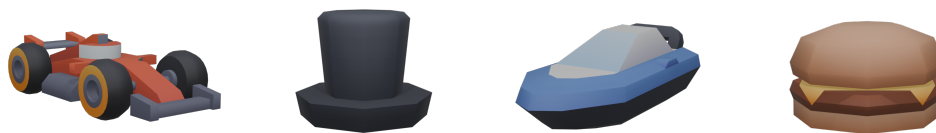


Figura 10: Diseño de algunos skins para las fichas de los jugadores en el juego Magnate.

5.2.4. Decisiones de diseño

Las primeras decisiones de diseño se tomaron sobre las **tecnologías** a usar en el proyecto. Las candidatas consideradas son todas tecnologías libres y sin licencia comercial. Para las tecnologías web se barajaron html con css y React. Aunque los miembros del equipo tenían experiencia con html y css, la dificultad de realizar un proyecto tan complejo en estos lenguajes nos llevó a decidirnos por React, cuya curva de aprendizaje es bastante menor una vez conocidas estas tecnologías, y cuenta con una forma fácil de incluir paquetes de componentes. Además, React cuenta con numerosas librerías sobre las que construir un juego. Se desestimó la librería para juegos boardgame.io ya que esta supliría el trabajo del que se encarga el backend, pero sí se utilizó Phaser para gráficos, componentes y elementos del juego, ya que el aprendizaje merecería la pena frente a realizar todo en React. Por motivos similares, se desestimó utilizar directamente HTML Audio para gestionar el sonido y se utilizó la librería Howler, que parecía más sencilla que otras del estilo. Como herramienta para la documentación, se barajaron react-docgen, TypeDoc y Storybook, y se decidió usar TypeDoc por ofrecer documentación automática para todos los archivos (clases, interfaces, funciones y módulos además de los componentes).

Como segundo cliente se decidió el escritorio sobre el móvil porque facilitaría las labores de interfaz, al no tener que conciliar diseños horizontales y verticales. Como nueva tecnología para el escritorio se optó por Godot, un motor orientado al desarrollo de juegos que tiene una interfaz para el desarrollo intuitiva y cuenta con portabilidad a una amplia gama de dispositivos, por lo que se presentaba como una buena herramienta para aprender. Otras tecnologías consideradas fueron Draft y Flutter, que se descartaron por no estar dirigidas a juegos. Para la herramienta para generación de documentación del proyecto en Godot se eligió Godoct por ser compatible con Godot 4, no como Doxygen, que tiene las mismas características, y por ser más automática que las demás consideradas (MkDocs y Sphinx) y bastar con seguir el estilo y forma de poner comentarios para obtener la documentación.

Como tecnologías de backend se barajaron Django, Flask, NextJS y FastAPI, y se decidió usar Django por tener experiencia previa en la tecnología. Además, salvo Flask, excesivamente sencilla, las tecnologías mencionadas ofrecen funcionalidades similares. Para la representación de la información se consideraron los formatos yaml, xml y JSON, y finalmente se decidió utilizar ficheros JSON por su uso extendido y generalizado, y la experiencia previa trabajando con ellos. Para la base de datos, se barajaron bases de datos SQL, que son con las que tenemos experiencia, y nos decantamos por PostgreSQL por ser compleja, potente y eficiente. La decisión de utilizar las herramientas automáticas de generación de documentación vienen por haberse encontrado tras una pequeña búsqueda, ver que estas bastarían para el proyecto, estar familiarizados con el formato docstrings, poder integrarse y generar pdfs de forma sencilla y que mkdocs es moderno

y fácil de configurar.

Otra decisión de diseño tomada fue utilizar el servidor de Mario Casado para el despliegue. Se optó por esta opción sobre cualquier otra plataforma online para el despliegue debido a su acceso gratuito y sin condiciones, y a la familiaridad de algunos de los miembros del equipo con dicho entorno.

La decisión sobre el diseño de la **arquitectura** viene directamente motivadas por las tecnologías empleadas y los flujos de trabajo habituales necesarios para desplegar aplicaciones que utilizan dichas tecnologías. Como servidor web sobre el que desplegar nuestro cliente web se decidió utilizar nginx en lugar de Apache, ya que contábamos con experiencia previa en dicho servidor y sus prestaciones son suficientes para el proyecto.

Sobre los **protocolos de comunicación** entre los componentes del sistema, se optó por una API Rest para las comunicaciones no relacionadas con el juego, ya que son más sencillas, y web sockets para poder almacenar el estado del juego y tratar las comunicaciones de frontend a backend (con los objetos responses) y de backend a frontend (con actions, enviando el contenido en un JSON serializado para facilitar el tratamiento en el frontend).

6. Memoria del proyecto

En este capítulo se describe el ajuste a la realidad de lo previsto en el plan de la Sección 3.

6.1. Inicio del proyecto

La elección inicial de tecnologías permitió que el proyecto avanzara a buen ritmo una vez nos familiarizamos con ellas, siguiendo el plan inicial.

6.2. Ejecución y control del proyecto

A día 15 de marzo, la comunicación interna se ha concentrado más en los grupos de mensajerías para los equipos reducidos (backend, cliente web y cliente escritorio) y en persona para la medición del progreso y la comunicación de la empresa al completo. Creemos que esto refleja las necesidades de las actividades que hemos estado realizando hasta el momento y el primer periodo de adaptación al trabajo en un grupo tan grande, y que conforme avance el proyecto usaremos el resto de herramientas que hemos preparado. Lo mismo sucede con las pruebas: de momento solo se han realizado tests automáticos para el backend, que han resultado muy útiles, y pruebas manuales en el frontend.

En cuanto al reparto de tareas, la duplicación del trabajo en frontend y ser la primera vez que trabajábamos juntos algunos de los integrantes hizo que nuestro planteamiento inicial del reparto de trabajo derivara en tener que repetir el trabajo por diferencias en diseños. Esto provocó que tuviéramos más trabajo durante un par de semanas, y nos retrasásemos dos semanas con respecto al plan inicial, que era demasiado optimista. Tomamos medidas el 27 de febrero cambiando progresivamente nuestra manera de repartir el trabajo en el frontend: en vez de asignar unas pantallas a web y otras a escritorio, decidimos asignarlas en primer lugar a web, ya que la plataforma es más restrictiva y limitada que escritorio, por si era necesario hacer algún pequeño cambio con respecto al diseño inicial consensuado, y después escritorio copiaría las pantallas. El cambio debido a esta divergencia se dio por completo el 13 de marzo, cuando revisamos el plan inicial, que tenía dos fases de integración y despliegue por requisitos, y lo cambiamos al plan actual, que se centra primero en el diseño e implementación y después en la integración y despliegue. El plan también favorece el reparto de los requisitos por persona, permitiendo el trabajo paralelo en mayor medida. Con estas medidas esperamos que no sucedan más retrasos.

Las herramientas y tecnologías empleadas, así como los procesos para conocer el trabajo realizado y pendiente y sus encargados, y los procesos para la implementación y procesos relacionados se han seguido respecto al plan, con resultados satisfactorios. Destacar que todavía no se ha tenido oportunidad de probar los procesos de integración ni despliegue.

La semana del 1 de abril se concretaron en el documento del plan las pruebas referentes a la integración y despliegue. También se decidió automatizar mediante Github actions la generación de la documentación del código, lo cual se implementó en la sesión de la práctica 5.

El 6 de abril, comparando diseños de web y escritorio, se decidió que la primera plataforma tuviera más confirmaciones y la segunda menos, para poder estudiar en un futuro qué resulta más entretenido a usuarios nuevos y habituales.

6.3. Cierre del proyecto

A día 15 de marzo, el proyecto sigue el plan descrito en la Figura 2, y los esfuerzos realizados están en torno al 40 % del proyecto, adecuado para el momento en el que nos encontramos.

Durante este tiempo se han implementado las estrategias de mitigación de riesgos como planteado. Los riesgos 3, 7, 9, 10 y 11 todavía no se han presentado, y los riesgos 1, 5, 8 y 4 se han presentado sin efectos negativos (aunque todavía están a tiempo de mostrarlos, sobre todo el último). El riesgo 6 se presentó muy temprano en el proyecto, en torno al 23 de febrero, y

fue solventado sin pasar más de una semana. Con respecto al riesgo 2, su aparición y efectos se explicaron en la subsección 6.2.

Además de los riesgos, surgió un pequeño imprevisto entre los miembros del equipo backend: al juntar código de distintos miembros en el entorno de desarrollo de Mario se veía algo de ruido en los ficheros (por cambio de formato y demás). Así que todos los miembros del equipo pasaron a usar el mismo entorno de desarrollo (VisualStudio, de los que ya estaban contemplados). No supuso un retraso de más de un día. Este imprevisto podría considerarse como riesgo en futuros proyectos, imponiendo como mitigación que las personas que editen los mismos ficheros lo hagan en el mismo entorno de desarrollo.

Hemos aprendido las siguientes lecciones:

- Usar el mismo entorno de desarrollo para editar los mismos ficheros evita problemas menores pero molestos.
- Al trabajar con distintas herramientas de forma que sea necesario replicar el trabajo puede ser una buena práctica ir más adelantado en una plataforma que en otra para evitar repetir trabajo, si una de estas tecnologías es más limitada que la otra.
- La comunicación en el equipo depende de las actividades que realicen, afectando tanto al contenido como a su forma.

El día 2 de abril resolvimos un malentendido entre el frontend y el backend por la interpretación distinta de una parte ambigua de las reglas. Ello supuso en torno a 1 hora de trabajo añadido para el backend, que por motivos de jugabilidad y esfuerzo se ajustaron a la versión propuesta por el frontend. Este es un ejemplo de manifestación del riesgo 6, ya que no definimos claramente esa parte de las reglas, que forma parte del requisito 1.

6.3.1. Dedicación por integrante

En la Tabla 5 recogemos las horas empleadas por cada miembro de la empresa en cada tarea a día 15 de marzo. Podemos comprobar que estas se corresponden con las tareas que tenían asignadas en un principio y están equilibradas teniendo en cuenta las actividades realizadas hasta la fecha. En total se han realizado 381,25 horas, que suponen en torno a un 40 % del proyecto.

7. Conclusiones

personales, mejorar los procesos... Además de conclusiones personales (razonadas) sobre el transcurso del proyecto realizado, es importante plantear ideas para mejorar los procesos llevados a cabo: si hubiera que iniciar un nuevo proyecto inmediatamente usando una metodología de gestión basada en procesos, ¿qué cambios haríais respecto a los procesos que habéis seguido durante este proyecto? ¿Qué cosas está claro que haríais de otra forma? ¿Qué cosas seguiríais haciendo más o menos igual? Pensad que si la respuesta es “no cambiaríamos nada, todo nos parece que estuvo bien”, lo que estáis transmitiendo es que no habéis aprendido nada en la asignatura, lo que no suele ser una buena señal.

Responsable	Horas
Hugo Naudín	43
Coordinación empresa	3,75
Coordinación backend	5
Lógica del juego	16,75
Bases de datos	4,25
Calidad	9
Serialización	4,25
Jaime Alconchel	35,5
Coordinación empresa	3,75
Coordinación backend	6,25
Lógica del juego	10
Bases de datos	10
Serialización	2
Documentación y diagramas	3
Calidad	0,5
Mario Casado	46
Coordinación empresa	2,75
Coordinación backend	4,25
Lógica del juego	19,5
Bases de datos	1,5
Serialización	8,5
Diseño de marca	1,5
Calidad	8
Miguel Royo	52,25
Coordinación equipo	3,25
Coordinación frontend	2
Diseño	13,5
Implementación	33,5
Cristina	52,25
Coordinación equipo	3,25
Coordinación frontend	2
Diseño	11,5
Implementación	35,5
Julia	59
Coordinación equipo	5,5
Coordinación frontend	1
Diseño	5,5
Implementación	13,5
Gamificación	4,5
Documentación	29,5
Nicolás	47,75
Coordinación equipo	3,25
Diseño	15,5
Implementación	28
Gamificación	0,5
Documentación y diagramas	0,5
Miguel Luquín	45,5
Coordinación equipo	1,5
Diseño	13
Implementación	30
Gamificación	1

Tabla 5: Dedicación: horas y tareas realizadas por los integrantes de la empresa.

Referencias

- [1] 08 - Roberta Williams. *Descripción de las heurísticas de los bots de Magnate*. Asignatura Proyecto Software (30226), documentación técnica de API. 2026. URL: https://github.com/UNIZAR-30226-2026-08/backend/blob/main/docs/agent_heuristics.md.
- [2] Godot Engine contributors. *Best practices*. Godot Engine official documentation. Godot Engine Documentation. 2024. URL: https://docs.godotengine.org/en/stable/tutorials/best_practices/index.html (visitado 14-03-2026).
- [3] Hasbro. *Monopoly: Reglas de juego (Edición en español)*. Accedido el 2 de febrero de 2026. URL: <https://www.hasbro.com/common/instruct/Monopoly%28Spanish%29.pdf>.
- [4] Newwby. *Godoct*. Repositorio de software. GitHub, 2024. URL: <https://github.com/newwby/Godoct> (visitado 14-03-2026).
- [5] Guido van Rossum, Barry Warsaw y Alyssa Coghlan. *PEP 8 – Style Guide for Python Code*. Python Enhancement Proposal. Python Software Foundation. 5 de jul. de 2001. URL: <https://peps.python.org/pep-0008/> (visitado 14-03-2026).
- [6] Kenney Vleugels. *Emotes Pack*. Paquete de recursos gráficos 2D (emotes) bajo licencia CC0. Kenney.nl. 2018. URL: <https://kenney.nl/assets/emotes-pack> (visitado 14-03-2026).

A. Resultados de la revisión intermedia

La revisión intermedia se recoge en el siguiente Excel que se muestra en la Figura 11.

Metodología del Trabajo y documentación	Respuesta (Sí/No)	Si la respuesta es que no, indicad cómo vais a mejorar en este aspecto. Si la respuesta es que sí, añadid información adicional que consideréis relevante.
Se lleva un registro escrito de las principales decisiones tomadas por el equipo.	Sí	
Los requisitos definitivos del proyecto han sido revisados por el profesor tutor y cuentan con su visto bueno.	Sí	
Hay un diagrama de navegación con los diseños de las interfaces de los frontends.	Sí	
Hay un diagrama de Gantt actualizado a fecha de hoy que refleja las fechas de inicio y fin reales de todos los requisitos ya terminados y la planificación de todos los requisitos pendientes.	Sí	Consultar índice de figuras y tablas
Las issues de GitHub incluyen todas las tareas necesarias para completar el proyecto (incluyendo tests, despliegue, documentación y gestión). Es especialmente crítico asegurarse de que estas tareas cubren el 100% de los requisitos.	No	Solo incluyen las tareas de las semanas inmediatas, se irán añadiendo conforme se acerque la fecha. Retroactivamente sí que se incluyen todas.
Todas las tareas de análisis, diseño, implementación, tests, despliegue y gestión del proyecto tienen un responsable conocido por todos.	Sí	Todas las issues de Github están asignadas.
Se está siguiendo una política de nombrado y gestión de documentos acordada (diseños, código, documentos de gestión...).	Sí	
Se está siguiendo alguna guía de estilo de codificación para generar código con un formato homogéneo.	Sí	
Se está siguiendo alguna guía de diseño de GUI para generar interfaces gráficas homogéneas y usables.	Sí	
Hay un diagrama de arquitectura actualizado con los componentes que forman los frontend y backend del sistema y sus interacciones.	Sí	
Hay un documento definiendo la API web que expone el backend.	Sí	https://unizar-30226-2026-08.github.io/backend/docs/doc_api.pdf
Hay documentación arquitectural adicional, describiendo otras vistas del sistema (módulos, incluyendo el modelo de datos y despliegue) y aspectos de la dinámica del mismo.	Sí	(Diagramas de la memoria)
Se está usando del sistema de gestión de incidencias de GitHub para asignar todas las tareas de desarrollo.	Sí	
Se está usando un proceso de control de versiones (workflow) que dicta cuándo, y en qué rama se sube código a cada repositorio de GitHub.	Sí	(Se trabaja en ramas locales y después se sube a la rama main).
Se están haciendo pruebas suficientes: unitarias, de integración, de sistema etc. Idealmente al menos una parte de estas pruebas, o todas, son automáticas.	Sí	Unirías si: casos de uso sistema 22+ casos de uso juego 45 frontend y backend (todas las transacciones posibles del diagrama de estados, prueba de serializadores, IA de 2 jugadores). Para la integración: websockets, cliente manual de websockets para simular la interacción con frontend. Faltan: prueba de partida del frontend, resto de pruebas de integración y de despliegue.
Se ha desplegado al menos una vez el sistema integrado, en su estado actual, en un entorno similar al que se prevea como entorno de despliegue final (entorno de producción). Idealmente este proceso se ha hecho ya varias veces, e idealmente este proceso se ha automatizado al menos parcialmente.	No	Se hará a partir del 15 de abril.
Estado del proyecto y grado de ejecución	Respuesta (Valor cuantitativo)	Mitigación de posibles problemas
Porcentaje de requisitos funcionales completados.	0	No hay ninguno completo porque todavía no hemos hecho integración ni despliegue, pero está la implementación por separado de todos los requisitos.
Horas de trabajo realizadas respecto de las previstas del total del proyecto.	0,677840909090909	Si este número es muy bajo, indicad qué vais a hacer para mejorarlo.
Estimación del porcentaje del proyecto completado, en desarrollo, y pendiente, en función de las tareas completadas, en desarrollo y pendientes.	65,00 %	Si el completado y/o el "en desarrollo" es poco, indicad qué vais a hacer para mejorarlo.
Actividad de cada integrante del equipo desarrollo. Repetid esta sección las veces que haga falta añadiendo el nombre de cada persona del equipo.	Respuesta (Valor cuantitativo)	Si algún integrante tiene grandes desfases respecto al resto, indicad la causa e indicad cómo vais a tratar de reconducir esta situación.
<i>Jaime Alconchel Gallego</i>		
Horas dedicadas al proyecto hasta este momento.	64,5	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	47	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	2288	
<i>Mario Casado Fontana</i>		
Horas dedicadas al proyecto hasta este momento.	66,75	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	62	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	2615	
<i>Cristina Embid Martínez</i>		
Horas dedicadas al proyecto hasta este momento.	82,25	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	80	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	35257	
<i>Hugo Naudín López</i>		
Horas dedicadas al proyecto hasta este momento.	74,75	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	62	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	3982	
<i>Miguel Ángel Luquín Guerrero</i>		
Horas dedicadas al proyecto hasta este momento.	73,25	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	31	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	48435	
<i>Miguel Ángel Luquín Guerrero</i>		
Horas dedicadas al proyecto hasta este momento.	73,25	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	31	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	48435	
<i>Julia Quero Pérez</i>		
Horas dedicadas al proyecto hasta este momento.	73	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	32	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	139572	
<i>Nicolás Eugenio de Rivas Morillo</i>		
Horas dedicadas al proyecto hasta este momento.	81,75	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	61	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	199331	
<i>Miguel Angel Royo Miguel</i>		
Horas dedicadas al proyecto hasta este momento.	80,25	
Horas previstas de dedicación al proyecto en total.	110	
Número de commits realizados en Github.	73	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	197012	
Enlace al vídeo de demostración del sistema		
https://drive.google.com/file/d/1zB9T_3uoDbt_CiUj4ZOCxvFutvWZ_n5l/view?usp=sharing		

Figura 11: Documento con la autoevaluación

B. Presentación final

Aquí hay que poner una copia de las transparencias usadas en la presentación final (pueden ir a un tamaño reducido, entre 2 y 4 transparencias por página), y explicar cómo esta presentación ha tenido en cuenta lo que habéis visto en los temas de teoría sobre cómo hacer presentaciones.

C. Diagramas

C.1. Diagrama de componentes y conectores.

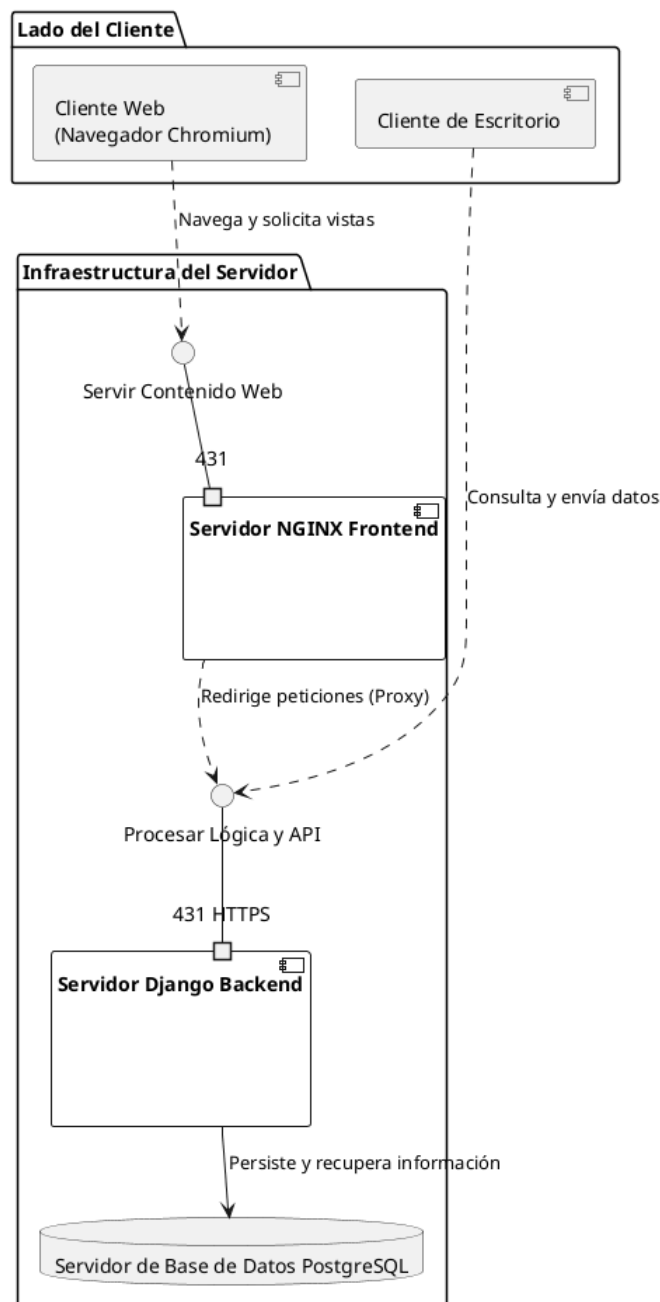


Figura 12: Diagrama de componentes y conectores.

C.2. Diagrama de estados de la fase de Subasta de un turno del juego Magnate



Figura 13: Diagrama de estados de la fase de Subasta de un turno del juego Magnate.

C.3. Diagramas de módulos

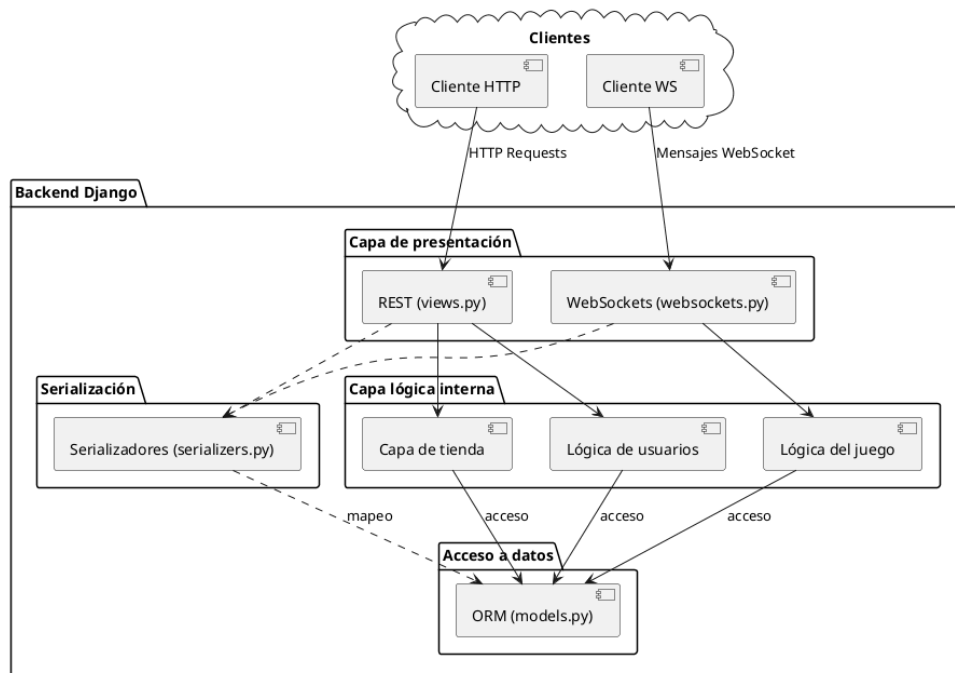


Figura 14: Diagrama de módulos del Backend.

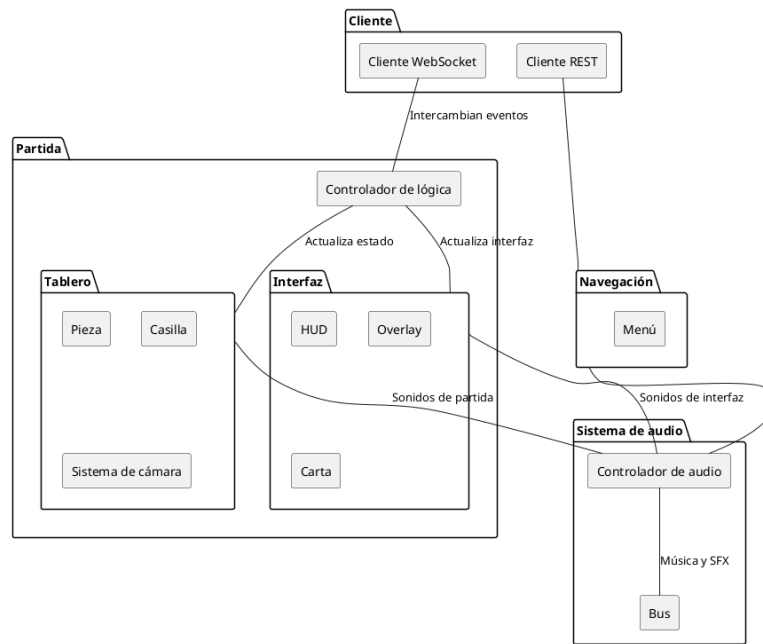


Figura 15: Diagrama de módulos del cliente de escritorio.

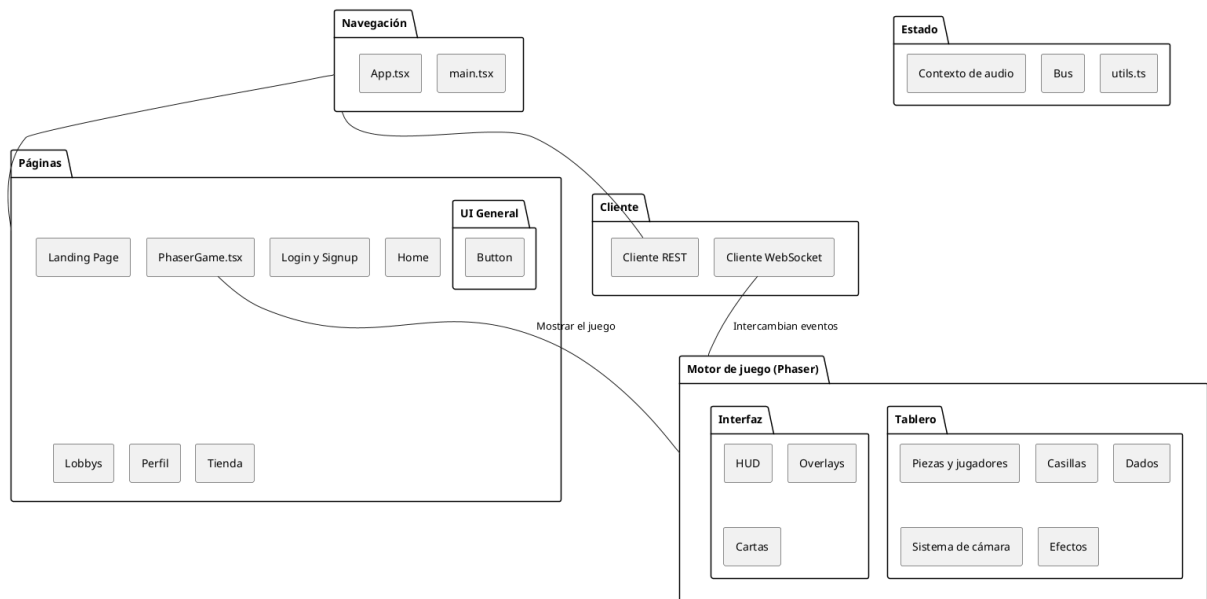


Figura 16: Diagrama de módulos del cliente web.

D. Análisis de riesgos

En la Figura 17 se recoge el análisis de riesgos detallado realizado por la empresa.

E. Reglas

E.1. Introducción

En este documento se definirán las mecánicas y reglas del juego *Magnate*. Se explicarán las diferentes fases del juego, las acciones que pueden realizar los jugadores y las condiciones de victoria.

Se definirá desde el diseño del tablero hasta los distintos eventos especiales que pueden ocurrir, considerando todos los casos posibles.

Las reglas están basadas en las reglas originales del Monopoly [3] tomando inspiración adicional de la saga de videojuegos Mario Party. *Magnate* busca ser un juego dinámico con alto grado de aleatoriedad, pero sin perder la estrategia del Monopoly original.

E.2. Objetivo del juego

El objetivo del juego es ser el jugador con mayor valor neto al final de la partida, limitada por un número de rondas. El valor neto de un jugador se calcula mediante la suma de su dinero en efectivo más el valor de venta de todas sus propiedades, sin contar las que tenga hipotecadas y el valor de venta de sus construcciones.

E.3. Equipamiento

A continuación está la lista de elementos para una partida.

- Tablero
- 2 Dados normales
- 1 Dado bus
- Cartas de las propiedades
- Casas
- Hoteles
- Cartas de evento fantasía

E.3.1. Los dados: definición y funcionamiento

En el juego hay 3 dados en juego, 2 normales de 6 caras numerados del 1 al 6 y uno especial llamado *Dado bus*. El Dado bus está compuesto de 6 caras, 3 de ellas con el dibujo de un bus y las otras 3 numeradas del 1 al 3.

En una tirada de dados se tiran siempre los 3 dados a la vez. En caso de salir 3 números se tendrá en cuenta para el movimiento la suma total de los 3. En caso de salir un bus el jugador podrá elegir 3 valores: el valor de uno de los dados normales, el valor del otro de los dados normales o la suma de los 2 dados normales.

Para las mecánicas de los dobles se tendrán en cuenta únicamente los dados normales. Al sacar dobles, el jugador gana una tirada extra. Si llega a sacar dobles una tercera vez consecutiva en el mismo turno, va directamente a la cárcel.

Si en la tirada sale un triple (en los casos en los que el Dado bus saque número), el jugador puede viajar a la casilla del tablero que quiera, aplicándose el efecto de caer en esa casilla tal y como hubiera sido si la tirada hubiera sido convencional. Después de sacar un triple no se vuelve a tirar (como si de unos dobles se tratara). Los triples no cuentan en la racha de dobles, por lo que no te pueden mandar a la cárcel.

E.3.2. Tablero

El tablero está compuesto de 2 pistas cuadradas concéntricas: el anillo exterior tiene 36 casillas y 4 esquinas y el anillo interior tiene 12 casillas y 4 esquinas. La particularidad de las casillas para transición y cómo afectan al tablero se comentará con las casillas de puentes.

E.4. Casillas

En esta sección se definirán todos los distintos tipos de casilla que hay en el tablero y sus efectos y mecánicas.

E.4.1. Propiedades

Las propiedades son casillas que se pueden comprar, vender e hipotecar, y por las que se cobra alquiler a jugadores rivales. Si un jugador cae en una de estas casillas pueden pasar varias cosas en función del estado de compra de la propiedad:

- La casilla no tiene dueño. En este caso el jugador tiene derecho a comprarla por el precio de compra indicado. En caso de no querer comprar o no tener suficiente dinero, la propiedad pasa a ser subastada.
- La casilla tiene dueño. El jugador debe pagar el alquiler calculado en función de las construcciones o de si tiene el grupo completo.
- La casilla está hipotecada. No ocurre nada.
- La casilla es del propio jugador. No ocurre nada.

E.4.2. Grupos de propiedades

Las propiedades están divididas en grupos de color. Un jugador que posea todas las propiedades de un mismo grupo, podrá cobrar el doble de alquiler por las propiedades sin construcciones y obtendrá derecho a construir en ellas.

E.4.3. Puentes

Los puentes son un tipo especial de propiedad. En el juego tienen una función doble: poder ganar dinero por alquiler (como el resto de propiedades), y habilitar la transición entre el cuadrado interior y el cuadrado exterior.

El alquiler a pagar en los puentes viene marcado por el número de puentes que el jugador posea.

La mecánica de transición viene marcada por el valor usado en la tirada de dados (las casillas que el jugador elige mover). Si la tirada es par se continúa en el mismo cuadrado, si es impar se cambia de cuadrado (o viceversa, lo que indique la casilla).

En caso de caer justo en la casilla del puente, la decisión par o impar la tomará la tirada de dados de la siguiente jugada y no la tirada de la jugada de caer en el puente.

Las casillas puente, a pesar de ocupar el espacio de 2 casillas convencionales únicamente contarán como una en los movimientos tanto en los casos de mantenerse en el cuadrado como en el caso de transicionar al otro cuadrado.

E.4.4. Servidores

Los servidores son otro tipo especial de propiedad. El valor de su alquiler viene determinado por la cantidad de servidores que el jugador posea.

E.4.5. Casillas fantasía

Al caer en una casilla fantasía, aparecerán 2 cartas, una desvelada y una oculta. La carta desvelada tendrá un evento fantasía aleatorio, que se podrá aplicar por el precio indicado en la carta. El evento fantasía de la carta oculta es gratuito. En caso de no querer comprar el evento desvelado, o no tener dinero para comprarlo, habrá que escoger la carta oculta obligatoriamente. En caso de comprar y aplicar el evento fantasía de pago, no se aplicará ni desvelará el evento oculto.

Los eventos fantasía tendrán acciones positivas (como ganar dinero), negativas (como perder propiedades) o neutras (como mover a otro jugador a cualquier casilla del mapa).

E.4.6. Vas a la cárcel

Si caes en esta casilla vas directamente a la cárcel y acaba tu turno. En el caso de que el jugador deba dinero al banco se le permitirá realizar las gestiones necesarias para saldar su deuda.

E.4.7. Cárcel

Un jugador en la cárcel tiene derecho al principio de su turno a tirar los 2 dados normales. Si saca dobles puede salir de la cárcel usando el valor de la tirada y jugar su turno con normalidad (pero sin repetir turno por ser dobles). Si no saca dobles, puede pagar una fianza de valor fijo. Si al tercer turno en la cárcel no ha salido, será obligado a pagar la fianza.

Los jugadores en la cárcel tienen derecho a cobrar alquileres y a realizar acciones de gestión en su turno, pero no pueden moverse por el tablero ni participar en las subastas.

Los turnos en los que un jugador acabe en la cárcel acaban cuando se entre en la cárcel sin tener oportunidad de hacer gestiones (excepto el caso en el que tenga que saldar deudas con el banco).

E.4.8. Salida

En esta casilla comienza la partida y cada vez que un jugador pasa por ella (al caer justo sobre ella o la primera tirada en la que la dejas atrás de la vuelta al cuadrado correspondiente) recibe una cantidad fija de dinero.

Si por algún motivo un jugador decide mantenerse en el cuadrado interior y no pasar por salida, no recibirá dinero.

E.4.9. Tranvías

Hay 4 estaciones de tranvía repartidas por las esquinas del tablero. Si un jugador cae en una de estas casillas podrá moverse directamente a otra estación de tranvía de su elección pagando un precio por el desplazamiento. Solo puede hacerse un desplazamiento de este tipo por turno.

E.4.10. Párking

Durante la partida, las multas, pagos de impuestos y demás ingresos a la banca en general, se acumularán en un bote. El jugador que caiga en el parking se llevará el bote completo. Si el bote está vacío, no ocurre nada.

E.5. Preparación

Cada jugador comienza con una cantidad de dinero determinada. El turno se elige tirando un único dado, el que obtenga mayor valor empieza. En caso de empate se vuelven a lanzar los dados para desempatar. El orden de valores indica el orden de juego de mayor a menor.

E.6. Turno de juego

Al comienzo del turno, el jugador lanzará los dados y moverá en función de lo que salga. Al caer en una casilla, se aplicará su efecto correspondiente. Tras todos los efectos de casilla, el jugador podrá realizar acciones de gestión o transacciones. Estas acciones son: administrar sus propiedades (comprar o vender construcciones, hipotecar o deshipotecar propiedades), realizar intercambios (comprar o vender propiedades a otros jugadores, ver la Sección E.10) y declararse en bancarrota (rendirse, ver la Sección E.12).

Haga lo que haga el jugador, su balance debe quedar positivo al final de su turno, de lo contrario será declarado en bancarrota y eliminado de la partida.

E.7. Subastas

Las subastas serán pujas abiertas de valor oculto. Es decir, cada jugador participante en la puja decidirá si participar o no, y en caso de participar elegirá el valor por el que puja. El resto de jugadores no sabrá en ningún momento los jugadores que deciden participar en la puja ni los valores por los que pujan hasta el final de la subasta, donde se desvelarán los valores y el ganador de la puja. En caso de empate la propiedad no se dará a nadie.

Los jugadores que provoquen una subasta no podrán participar en ella.

E.8. Alquileres

Si un jugador cae en una propiedad de otro jugador, deberá pagar el alquiler correspondiente. El alquiler se calcula en función de si el dueño tiene el grupo de propiedades completo y de las construcciones que haya en la propiedad.

En caso de que el jugador no tenga dinero para pagar el alquiler al dueño, el banco pagará el alquiler y al final del turno, el jugador deberá saldar su deuda con el banco mediante el hipotecado de propiedades o la venta de construcciones o propiedades tanto al banco como a otros jugadores.

E.9. Construcciones

Una vez un jugador tiene el grupo completo de un color, podrá construir casas en las propiedades de este grupo. El coste de construcción de cada casa vendrá marcado en cada propiedad. Las casas deben construirse de forma escalonada, es decir, si se quiere construir una segunda casa en una propiedad primero se tendrá que tener una casa en el resto de propiedades del grupo.

Una vez un jugador tenga 4 casas en todas las propiedades de un grupo, podrá sustituir las 4 casas de una propiedad por un hotel por el precio de compra de una casa.

A la hora de demoler construcciones se hará igualmente de forma escalonada en sentido inverso. En el caso particular de los hoteles, no se demolerán de golpe, sino que primero se pasará de hotel a 4 casas. El jugador recibirá la mitad del precio de compra de las construcciones demolidas.

E.10. Intercambios

Cada turno, el jugador podrá proponer intercambios a otros jugadores. Un intercambio se hará entre el jugador que lo propone y otro que podrá elegir. Se podrán intercambiar dinero y cartas (propiedades, servidores y puentes) por dinero y cartas.

No se podrán intercambiar propiedades de un grupo de propiedades en el que alguna de las propiedades tengan construcciones.

E.11. Bonificaciones finales

Al final de la partida, antes de desvelar al ganador, se aplicarán 3 bonificaciones finales aleatorias. Las bonificaciones darán una cantidad de dinero determinada por un porcentaje del valor total de todos los jugadores sumado. Estas bonificaciones vienen determinadas por estadísticas de la partida, como el jugador que más se haya desplazado o el que más alquileres ha tenido que pagar al resto de jugadores, entre otras.

E.12. Bancarrota

Si un jugador no logra acabar su turno con balance positivo, incluso después de hipotecar sus propiedades, de vender todas sus construcciones y en general después de hacer gestiones para recaudar dinero, todas las propiedades que le queden serán embargadas por el banco, poniéndolas inmediatamente a la venta por su valor original.

Esto se aplicará también a los jugadores que se rindan, cuyo dinero en metálico quedará además absorbido por el banco sin ir al parking gratuito. Un jugador puede rendirse (declarándose en bancarrota) como parte de las transacciones en las fases de su turno posteriores a tirar los dados.

Índice de figuras

1.	Estructura del repositorio de despliegue	7
2.	Diagrama de Gantt con la planificación del proyecto.	12
3.	Diagrama de la arquitectura a alto nivel.	15
4.	Diagrama de estados de un turno de juego con sus fases.	16
5.	Diagramas de secuencia de las fases de negocios (izquierda) y liquidación (derecha).	17
6.	Diagramas de despliegue. A la izquierda, diagrama en 2 capas, como se realizará el despliegue, y a la derecha, en 4 capas, señalando las posibilidades de despliegue.	17
7.	Mapa de navegación (menús).	18
8.	Mapa de navegación (juego, acciones y negocios).	19
9.	Mapa de navegación (juego, caer en casillas y flujos).	20
10.	Diseño de algunos skins para las fichas de los jugadores en el juego Magnate.	21
11.	Documento con la autoevaluación	27
12.	Diagrama de componentes y conectores.	28
13.	Diagrama de estados de la fase de Subasta de un turno del juego Magnate.	29
14.	Diagrama de módulos del Backend.	29
15.	Diagrama de módulos del cliente de escritorio.	30
16.	Diagrama de módulos del cliente web.	30
17.	Análisis de riesgos.	32

Índice de cuadros

1.	Relación de cargos y tareas principales en la empresa 08 – Roberta Williams.	4
2.	Requisitos funcionales del sistema.	15
3.	Requisitos no funcionales del sistema.	15
5.	Dedicación: horas y tareas realizadas por los integrantes de la empresa.	25